计算机与软件学院毕业设计（论文）中期情况检查表

<table>
<tr><td>学院</td><td>计算机与软件学院</td><td>专业</td><td colspan="3">计算机科学与技术</td></tr>
<tr><td>学生姓名</td><td>黄静嘉</td><td>学号</td><td>2022280381</td><td>指导教师</td><td>李俊杰</td></tr>
<tr><td>设计（论文）题目</td><td colspan="5">基于 LLM 驱动的 text2cypher 图数据分析关键技术研究</td></tr>
<tr><td rowspan="2">目前已完成任务</td><td colspan="5">1、环境搭建与基准复现：完成 PyTorch/Transformers 开发环境配置；成功复现 Bird 数据集评测流水线，掌握 EX（执行正确率）等核心指标的量化计算方法。
2、多模型部署与 Schema 对标实验：完成 Qwen2.5-Coder、Mistral、DeepSeek-Coder 等基座模型的本地部署；通过通过移除与保留 Schema 描述进行对比实验，分析了 Schema 描述信息对 SQL 生成质量的影响，验证了结构化信息的重要性。
3、微调技术链路攻克：深入研究 SFT 与 RL 原理，熟练掌握 LLaMA-Factory 工具链；实操完成 LoRA 微调、模型量化及权重合并全流程，为大规模模型优化奠定了技术基础。
4、数据增强与负样本设计：精读 OmniSQL 等前沿文献，设计了针对本项目需求的指令微调数据集方案；探索利用“表/列识别错误”案例构造负样本，初步形成数据增强策略。
5、评测优化与 Agent 架构探索：实现基于哈希比对的高效验证方法，显著提升评测速率；搭建 ″生成-执行-判别″ Agent Pipeline，初步验证了多阶段生成策略对准确率的增益。
6、工程框架预研：学习 LangChain4J 及 RAG 技术原理，利用 LangGraph 完成原型 Agent 开发，积累了智能体逻辑编排与状态管理经验，为后期系统集成做好技术储备。</td></tr>
<tr><td colspan="5">是否符合任务书要求进度：符合☑　　不符合☐</td></tr>
<tr><td rowspan="2">尚需完成的任务</td><td colspan="5">1、完成基座模型的深度微调训练：基于大语言模型（如 Qwen-14B）进行 LoRA 微调，重点优化复杂多表连接、嵌套查询、聚合函数等场景的 SQL 生成准确性；利用错误数据构造高质量指令微调数据集，降低重复错误率。
2、实现面向垂直领域的迭代优化闭环技术：设计反馈收集机制，根据 SQL 执行错误自动定位问题并生成针对性训练样本；研究数据增强策略与模型再训练流程，构建可自我迭代的 Text-to-SQL 优化系统。
3、进行系统集成：搭建完整的 Text-to-SQL 原型系统，整合自然语言理解、Schema 解析、SQL 生成、执行验证等核心模块，支持自然语言到 SQL 的转换、执行与结果展示。
4、完成演示系统原型开发与整体性能评估：在 Spider、Bird 等公开基准数据集上进行端到端性能测试，与 SOTA 方法进行对比分析；完成用户交互界面开发，支持可视化查询与结果展示。
5、完成毕业论文撰写：撰写包含研究背景、相关工作、方法设计、实验结果与分析、总结与展望等章节的毕业设计论文，总字数不少于 15000 字。</td></tr>
<tr><td colspan="5">能否按期完成设计（论文）：能☑　　不能☐</td></tr>
<tr><td>存在的问题及拟采</td><td colspan="5">存在的问题：（不少于 200 字）
1、评测效率与准确性的冲突：在实验迭代中，简单的哈希值比对法虽然快，但无法处理“逻辑等价但写法不同”的 SQL，导致评测结果偏低；而全量执行验证在 Bird 等大数据集上耗时过长，严重拖慢了模型微调后的验证周期，急需一种兼顾</td></tr>
</table>

<table>
<tr><td rowspan="2">取的办法</td><td>速度与精度的评测方案。
2、复杂数据库结构的理解偏差：在处理 Bird 等包含大量表和模糊字段名（如缩写、专业术语）的数据库时，模型常出现“张冠李戴”的情况。单纯依靠 Prompt 传入 DDL 信息，模型仍会产生幻觉，生成不存在的列名或错误的外键关联，导致生成的 SQL 无法执行。
3、系统集成中的响应超时与鲁棒性问题：由于 14B 规模的模型推理耗时较长（通常需数秒），导致前端网页常因等待响应而出现假死或超时报错。此外，当模型生成语法错误的 SQL 时，系统往往直接返回后端报错代码，缺乏对非技术用户的友好提示和自动纠错能力，演示系统的稳定性有待加强。</td></tr>
<tr><td>拟采取的办法：（不少于 200 字）
1、建立分级评测验证机制：针对不同复杂度的查询，开发一套“哈希初步过滤 + 抽象语法树（AST）深度比对”的混合方案。对于简单查询利用哈希快速过检，对于复杂聚合查询则解析 SQL 的语法树结构，判断其逻辑是否与标准答案等价，从而在不增加过多耗时的前提下提升评测的公信力。
2、优化 Schema 描述与检索策略：引入 RAG（检索增强生成）思路，将数据库的元数据和字段注释存入向量库。在提问前，先检索出最相关的表结构片段，并在 Prompt 中加入 2-3 条真实的示例数据（Sample Rows）。这种“看样学样”的方式能有效帮助模型识别字段含义，纠正缩写歧义，降低映射错误率。
3、采用轻量化前后端分离架构：前端使用 Vue 实现交互界面，通过 Axios 异步调用基于 FastAPI 搭建的后端接口，解决推理耗时导致的界面阻塞问题。引入简单的异常截获逻辑，当 SQL 执行失败时，不再直接报错，而是将错误信息反馈给模型尝试“一次自修复”或向用户返回直观的错误提示；同时利用本地缓存减少重复查询的等待时间，确保演示流程顺畅。</td></tr>
<tr><td>指导教师建议与意见</td><td>1、核心逻辑的验证深度：在解决“多解 SQL”评测难点时，AST（语法树）比对虽然比哈希值更精准，但在处理复杂的 JOIN 关联顺序时仍可能有局限。建议在论文中增加一小组人工抽样比对实验，量化分析 AST 相比传统哈希比对在准确率上的具体提升，这能显著增强你实验结论的说服力。
2、数据增强的针对性：针对 Schema 映射偏移问题，引入 RAG 和 Sample Rows（示例行）是非常务实的做法。建议在后续微调 14B 模型时，专门构造一批包含“缩写字段”和“多表同名字段”的干扰样本进行强化训练。通过这种“错题集”式的微调，能有效观察模型在垂直领域下的鲁棒性变化，是论文中非常好的创新点分析。
3、原型系统的交互闭环：在前后端分离的开发中，不要仅满足于“输入自然语言-输出表格”。建议增加一个“执行逻辑解析”展示位，将 Agent 推理的中间过程（如：它识别出了哪些表、连接条件是什么）简要展示给用户。这不仅能降低由于模型推理慢带来的焦虑感，也能在 SQL 执行报错时，让用户直观看到是哪个环节出了问题，体现出系统设计的完整性。
4、进度管理与文档规范：目前中期进度符合要求，但后续涉及 14B 模型的微调与多轮评测，算力资源与时间成本较高。建议严格执行实验日志记录，保存好不同版本模型的对比参数，为最后的毕业论文提供详实的图表支撑。

签字：李绘志　　2026 年　03 月 10　日</td></tr>
</table>